CORE JAVA TRAINING · PART 3 OF 25

Control Flow & Input

Decisions, repetition and user input, without the silent bugs

Difficulty	Easy
Prepared for	Mitansh Kanani
Starts with	Revision page: your Part 2 mistakes
Covers	if/else traps · dangling else · classic and arrow switch · switch expressions
	while · do-while · for · loop traps · break, continue, labels
	Scanner · the nextInt/nextLine trap · input validation · classic loop programs
Assignments	8 tasks + 4 pending and 1 optional from Part 2
Tested on	JDK 21 (all code, outputs and error messages verified)

Contents

0	Revision: mistakes from Part 2
1	if and else
2	switch
3	Loops
4	break, continue and labels
5	Reading input with Scanner
6	Classic loop programs
7	Common errors in this part
8	Self-check questions
9	Cheat sheet
10	Assignments

Until now your programs ran top to bottom, once. Part 3 gives them **decisions** (if, switch), **repetition** (loops) and **input** from the user (Scanner). With these three, you can write real interactive programs, like the menu-driven ATM at the end of this part. Almost every bug in this topic compiles fine, so learn to **trace** code in your head: which branch runs, how many times a loop runs, and what the values are after each step.

> **Note**
> Every example, output and error message in this PDF was compiled and run on JDK 21. Scanner examples were run with real typed input.

0 Revision: mistakes from Part 2

Read this before anything else. Each row is something that actually happened in your Part 2 assignments, with the idea to keep.

What happened	What to remember
You printed char's SIZE, MIN_VALUE and MAX_VALUE with `%c` and got `?` (A1).	`%c` prints a value **as a character**. 16, `'\u0000'` and `'\uFFFF'` are not printable, so the terminal shows ?. For the numbers use `%d` and cast: `(int) Character.MAX_VALUE`. A char passed straight to `%d` throws `IllegalFormatConversionException`.
You printed `Double.MIN_VALUE` as double's maximum (A1).	`Double.MIN_VALUE` is the **smallest positive** double, `4.9E-324`. The maximum is `Double.MAX_VALUE`; the most negative is `-Double.MAX_VALUE`.
You predicted `char c4 = 'a' + 1;` would not compile (A2).	It compiles. `'a' + 1` is a **compile-time constant** (98) that fits in a char, so no cast is needed. With a variable, `char ch = 'a'; char c = ch + 1;` fails. Printed directly, `'a' + 1` is `98` (char + int = int).
You couldn't explain why `(int) 3e9` and `(int) 2_147_483_648L` differ (A2).	The starting type decides the rule. `3e9` is a **double**: decimal to int **saturates** at `2147483647`. The other is a **long**: integer narrowing keeps the low 32 bits, and 2^{31} in 32 bits is just the sign bit, so `-2147483648`.
You left the byte and short narrowing predictions blank (A2).	Shortcut: for byte, if the value is above 127 subtract 256, below -128 add 256. `(byte) 200` = 200 - 256 = `-56`. For short, use 65536: `(short) 40000` = `-25536`.
You parsed `args[0]` again with `Integer.parseInt`; input `36.6` crashed with `NumberFormatException` (A4).	Convert input **once**, then work with the variable. You already had `double celsius`, so `int cel = (int) celsius;` never crashes. `Integer.parseInt` only accepts whole-number text.

Carried-over assignments

Pending (Part 2): A5 IncrementPuzzle, A6 BitwiseToolkit, A7 SwapThreeWays, A8 ShortCircuitProof. These are C1–C4 in Section 10.

Optional (Part 2): A3 NumberTraps. This is C5 in Section 10.

Done: B1 Documented.java (Javadoc).

1 if and else

1.1 The basic forms

```
if (marks >= 40) {
    System.out.println("Pass");
}

if (marks >= 40) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
```

The condition inside the brackets must be a **boolean** expression: something that is exactly `true` or `false`. Unlike C, Java does not accept a number as a condition.

```
int x = 5;
if (x = 5) { }
```

javac

```
error: incompatible types: int cannot be converted to boolean
```

You wrote `=` (assignment) instead of `==` (comparison). `x = 5` produces an int, so Java stops you. Good. But there is one case where it does **not** stop you:

The boolean assignment trap

```
boolean done = false;
if (done = true) {
    System.out.println("Runs!");   // prints Runs!
}
```

`done = true` assigns true to done and the whole expression is a boolean (true), so it compiles and always runs. The fix and the professional style: never compare booleans with `==`. Write `if (done)` and `if (!done)`.

1.2 The else-if ladder

```
if (marks >= 90) {
    grade = "A";
} else if (marks >= 75) {
    grade = "B";
} else if (marks >= 60) {
    grade = "C";
} else {
    grade = "F";
}
```

Java checks the conditions **from top to bottom** and runs only the **first** branch whose condition is true. The rest are skipped, even if they are also true. So the order matters:

Wrong order: the general check swallows the specific one

```
int marks = 72;
if (marks >= 60) {
    System.out.println("C?");
} else if (marks >= 75) {
    System.out.println("never reached");   // 75+ is also >= 60, so it never gets here
}
```

Trainer's tip

In a ladder of ranges, go from the most restrictive to the least (90, 75, 60), or the other way round with `<`. Then each branch only needs one comparison, because the earlier branches already ruled out the rest.

1.3 Braces: the indentation lie

```
int marks = 30;
if (marks >= 40)
    System.out.println("Pass");
    System.out.println("Congratulations!");
```

Output

```
Congratulations!
```

Without braces, `if` controls only the **one** statement right after it. The second println is outside the if, whatever the indentation says. Java ignores indentation completely.

Common mistake

Always use braces, even for one line. Adding a second line later is how this bug gets into real code. Most company style guides make braces mandatory.

1.4 The semicolon trap

```
int x = 3;
if (x > 5); {
    System.out.println("Always prints");
}
```

The `;` right after the condition is an **empty statement**. The if controls that empty statement, and the block in braces is just a normal block that always runs. It compiles without any error. The same trap exists for `for` and `while` loops.

1.5 Nested if and the dangling else

```
int a = 5, b = 15;
if (a > 10)
    if (b > 10)
        System.out.println("both big");
else
    System.out.println("a is small");
System.out.println("end");
```

Output

```
end
```

You might expect “a is small”, because a is 5. But an `else` always belongs to the **nearest unmatched if**, here `if (b > 10)`. Since the outer if is false, nothing inside runs at all. The indentation is misleading. With braces, there is no question:

```
if (a > 10) {
    if (b > 10) {
        System.out.println("both big");
    }
} else {
    System.out.println("a is small");
}
```

Your evaluator might ask

Q: What is the dangling else problem?

When nested ifs have only one else and no braces, it is ambiguous which if the else belongs to. Java resolves it by attaching the else to the nearest if. Using braces removes the ambiguity.

1.6 if/else and definite assignment

Remember from Part 2: the compiler checks every path before letting you read a local variable. An else-if chain with no final `else` has a path where nothing is assigned:

```
int x;
if (args.length > 0) x = 1;
else if (args.length == 0) x = 2;
System.out.println(x);
```

javac

```
error: variable x might not have been initialized
```

You know these two conditions cover every case, but the compiler does not reason about values. It only sees an if-else-if with no plain `else`. Make the last branch a plain `else`.

1.7 if vs the ternary operator

```
String result = marks >= 40 ? "Pass" : "Fail";            // choosing a VALUE: ternary is fine
String g = marks >= 90 ? "A" : marks >= 75 ? "B" : "C";   // chained: legal, but hard to read
```

Use the ternary when you are choosing between two **values**. Use `if` when you are choosing between **actions** (statements), or when there are more than two outcomes.

2 switch

2.1 The classic switch statement

```
switch (day) {
    case 1:
        System.out.println("Mon");
        break;
    case 2:
        System.out.println("Tue");
        break;
    default:
        System.out.println("Other");
}
```

Java evaluates `day` once, **jumps** to the matching `case` label, and starts running from there. `break` exits the switch. `default` runs when no case matches. It is optional, and it can be placed anywhere, though by convention it goes last.

2.2 Fall-through

A case label is only an **entry point**, not a wall. Without `break`, execution continues into the next case's code, and the next, until a break or the end of the switch:

```
int day = 2;
switch (day) {
    case 1: System.out.println("Mon");
    case 2: System.out.println("Tue");
    case 3: System.out.println("Wed");
    default: System.out.println("Other");
}
```

Output

```
Tue
Wed
Other
```

Forgetting `break` is the most common switch bug. But fall-through is sometimes used on purpose, to give several cases the same code:

Intentional fall-through

```
switch (grade) {
    case 'A':
    case 'B':
        System.out.println("Good");      // runs for A and for B
        break;
    case 'C':
        System.out.println("Okay");
        break;
    default:
        System.out.println("Try harder");
}
```

2.3 What can you switch on?

Allowed	Not allowed
`byte`, `short`, `char`, `int` and their wrappers (Byte, Short, Character, Integer)	`long`, `float`, `double`, `boolean`
`String` (Java 7+)	Arrays, or most other objects (until pattern matching, Java 21)
`enum` values (Java 5+)	

```
long x = 5;
switch (x) { case 5: break; }
```

javac

```
error: selector type long is not allowed
```

Rules for the case labels:

- Each label must be a **compile-time constant**: a literal or a `final` constant. A normal variable gives `error: constant expression required`.
- No duplicates: `error: duplicate case label`.
- The label must fit the selector's type. `case 300:` does not compile when switching on a byte.

Under the hood

Why only constants? The compiler turns a switch into a **jump table**. With `javap -c` you can see it:

Close-together cases (1, 2, 3) become a `tableswitch`: the value is used directly as an index into a table of addresses. That is one step, no matter how many cases there are.

Spread-out cases (1, 100, 5000) become a `lookupswitch`: a sorted list searched quickly. Both need every label value known at compile time.

A String switch first switches on `s.hashCode()`, then calls `equals()` to confirm the match. That is why switching on a `null` String throws `NullPointerException`: you cannot call hashCode() on null.

Switch on a String

```
String cmd = "stop";
switch (cmd) {
    case "start": System.out.println("Starting"); break;
    case "stop":  System.out.println("Stopping"); break;   // matches: Strings compared with equals()
    default:      System.out.println("Unknown");
}
```

2.4 The arrow switch (Java 14+)

```
switch (cmd) {
    case "start" -> System.out.println("Starting");
    case "stop"  -> System.out.println("Stopping");
    case "pause", "wait" -> System.out.println("Waiting");   // several labels, comma-separated
    default -> {
        System.out.println("Unknown command");               // braces for more than one statement
        System.out.println("Try start or stop");
    }
}
```

- **No fall-through.** Only the matching case runs, so you never need `break`.
- Several values share one case with commas, instead of stacking empty cases.
- You cannot mix `case X:` and `case X ->` in one switch: `error: different case kinds used in the switch`.

2.5 Switch expressions: switch that returns a value

```
int day = 6;
String type = switch (day) {
    case 1, 2, 3, 4, 5 -> "Weekday";
    case 6, 7 -> "Weekend";
    default -> "Invalid";
};                                  // note the semicolon: this is one assignment statement
System.out.println(type);           // Weekend
```

A switch **expression** produces a value, like the ternary operator but with many branches. Two extra rules apply:

Rule 1: it must cover every possible value

```
String n = switch (d) { case 1 -> "Mon"; case 2 -> "Tue"; };
```

javac

```
error: the switch expression does not cover all possible input values
```

An int has billions of possible values, so you need a `default`. (For an enum, listing every constant is enough.)

Rule 2: a block must yield its value

```
int days = switch (month) {
    case 2 -> {
        boolean leap = false;
        yield leap ? 29 : 28;       // yield = "this block's result is..."
    }
    case 4, 6, 9, 11 -> 30;
    default -> 31;
};
```

A single expression after `->` is the value automatically. A block in braces must say which value it produces with `yield`. Forget it and you get `error: switch rule completes without providing a value`. `yield` also works in old-style `case 2: yield 28;` switch expressions.

Your evaluator might ask

Q: What is the difference between break and yield?

`break` exits a switch **statement** (or a loop) without producing a value. `yield` ends a branch of a switch **expression** and gives its value.

2.6 if-else or switch?

Use switch when...	Use if-else when...
Comparing **one** value against several **exact constants** (menu choices, day numbers, commands).	Checking **ranges** (`marks >= 75`), several different variables, or complex conditions with && and ||.
You want a value from many options: a switch expression is clean and exhaustive.	The selector is a long, double or boolean.

3 Loops

A loop repeats a block of code. Every loop has the same four ingredients, even if some are hidden: a **start** state, a **condition** to keep going, the **body**, and an **update** that moves toward the end. If the update never makes the condition false, the loop never ends.

3.1 while

```
int count = 3;
while (count > 0) {
    System.out.println("Countdown: " + count);
    count--;                        // update: without this, the loop never ends
}
System.out.println("Liftoff!");
```

The condition is checked **before** each round. If it is false at the very start, the body runs **zero** times. Use while when you don’t know in advance how many rounds you need: keep reading until valid input, keep dividing until the number is 0.

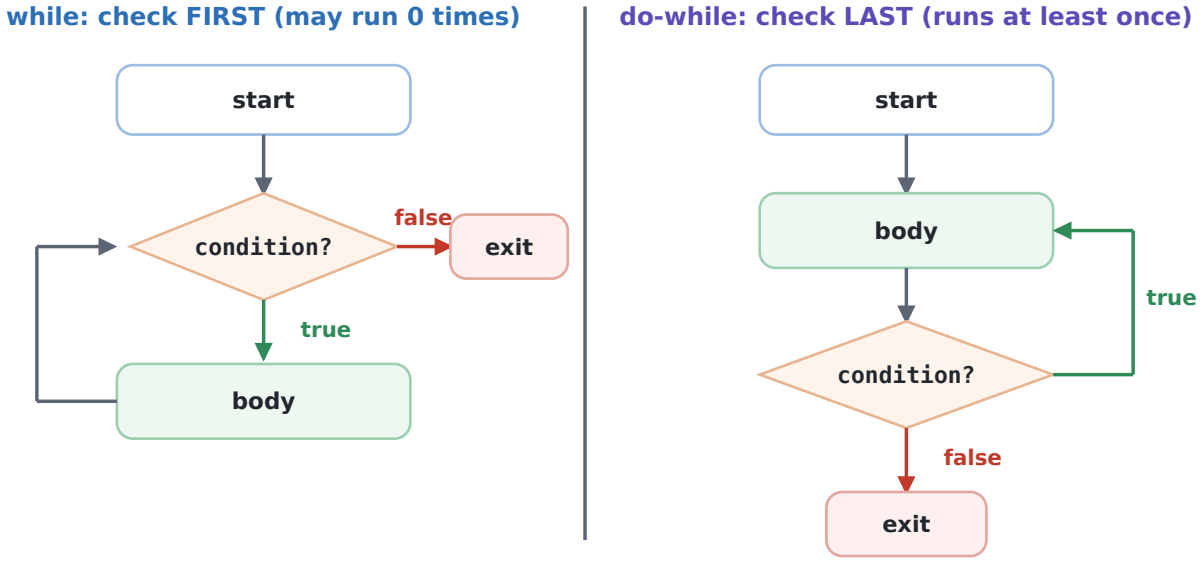
3.2 do-while

```
int i = 10;
do {
    System.out.println("do-while body runs once, i=" + i);
} while (i < 5);                    // the semicolon here is required
```

Output

```
do-while body runs once, i=10
```

The condition is checked **after** each round, so the body always runs **at least once**, even though `10 < 5` is false. Forgetting the semicolon after `while (...)` gives `error: ';' expected`.

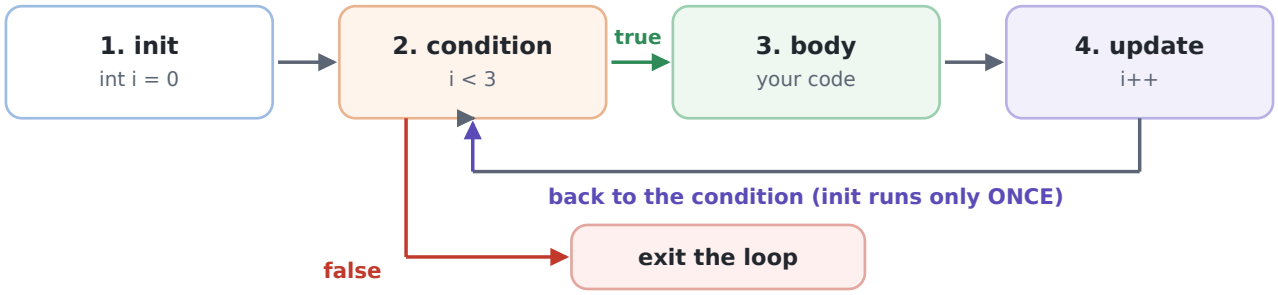


> **Trainer's tip**
> do-while is perfect for **menus**: you always want to show the menu at least once, and repeat until the user chooses Exit. You will build exactly this in assignment A8.

3.3 for

A for loop puts all the loop machinery on one line. Use it when you know how many times to repeat, or when you are counting through a range.

for (int i = 0; i < 3; i++) { body }



1. **init** runs once, before anything else.
2. **condition** is checked. If false, the loop ends immediately.
3. **body** runs.
4. **update** runs, then back to step 2.

```
for (int i = 1; i <= 5; i++) {
    System.out.print(i + " ");                    // 1 2 3 4 5
}

for (int i = 10; i >= 0; i -= 2) { ... }          // count down in steps of 2
for (char c = 'a'; c <= 'e'; c++) { ... }         // abcde: chars are numbers (Part 2)
for (int a = 0, b = 10; a < b; a += 3, b -= 3) {  // two variables: a:b = 0:10, 3:7
    System.out.print(a + ":" + b + " ");
}
```

Scope of the loop variable

```
for (int i = 0; i < 3; i++) { }
System.out.println(i);
```

javac

```
error: cannot find symbol
  symbol:   variable i
```

A variable declared in the for header exists only inside the loop. If you need its final value afterwards, declare it before the loop: `int i; for (i = 0; i < 3; i++) { }`.

> **Note**
> All three parts of a for header are optional. `for (;;) { }` is an infinite loop, the same as `while (true) { }`. There is also a **for-each** loop, `for (int x : numbers)`, for going through arrays and collections. You will learn it with arrays in Part 4.

3.4 Infinite loops and unreachable code

```
while (true) {
    // ... must contain a break or return, or it runs forever
}
```

Infinite loops are normal in servers and menus, as long as there is a `break` inside. The compiler also knows a `while (true)` with no break never finishes, so code after it is an error:

```
while (true) { }
System.out.println("x");
```

javac

```
error: unreachable statement
```

And here is a famous inconsistency:

```
while (false) { System.out.println("x"); }   // error: unreachable statement
if (false) { System.out.println("x"); }      // compiles fine!
```

> **Your evaluator might ask**
> **Q: Why does while(false) give an error, but if(false) does not?**
> The Java Language Specification makes a deliberate exception for `if`, so developers can switch code on and off with a constant: `static final boolean DEBUG = false; if (DEBUG) { ... }`. The compiler simply leaves that block out of the bytecode. Loops get no such exception, so a loop body that can never run is reported as unreachable.

3.5 Choosing the right loop

Loop	Checks condition	Runs at least once?	Best for
while	before each round	No (0 or more)	Unknown number of rounds: processing digits, waiting for valid input

Loop	Checks condition	Runs at least once?	Best for
do-while	after each round	Yes (1 or more)	Menus, “ask once, repeat if needed”
for	before each round	No (0 or more)	Counting: a known number of rounds or a range
for-each	(automatic)	No	Every element of an array or collection (Part 4)

3.6 Loop traps

Trap 1: decimal loop counters

```
int count = 0;
for (double x = 0.0; x < 1.0; x += 0.1) count++;
System.out.println(count);           // 11, not 10!
```

After adding 0.1 ten times, x is `0.9999999999999999`, which is still less than 1.0, so there is an 11th round. That is the floating-point imprecision from Part 2. Worse, `x != 1.0` as the condition would **never** become false: an infinite loop. Always count with an int and compute the decimal: `for (int i = 0; i < 10; i++) { double x = i * 0.1; }`.

Trap 2: overflow in the counter

```
for (byte b = 0; b < 200; b++) { }   // infinite loop!
```

A byte can never reach 200. After 127, `b++` wraps to -128 (Part 2), so `b < 200` is always true. The compiler doesn’t warn you.

Trap 3: off-by-one

```
for (int i = 0; i < 5; i++)    // runs 5 times: 0,1,2,3,4
for (int i = 1; i <= 5; i++)   // runs 5 times: 1,2,3,4,5
for (int i = 0; i <= 5; i++)   // runs 6 times: 0..5  <- usually a bug
```

> **Trainer’s tip**
> To count rounds quickly: with `<`, it is end - start. With `<=`, it is end - start + 1. Most loops start at 0 and use `<`, because that matches array indexes (Part 4).

Trap 4: the semicolon after the header

```
for (int k = 0; k < 3; k++); {
    System.out.println("Hi");        // prints once, not three times
}
```

Same as the if trap in Section 1.4: the loop repeats an empty statement three times, then the block runs once.

4 break, continue and labels

4.1 break and continue

Keyword	Inside a loop it...	Inside a switch it...
`break`	Exits the loop immediately. Execution continues after the loop.	Exits the switch.
`continue`	Skips the rest of **this** round and jumps to the next round (for a for loop, the update runs first).	Not allowed on its own (only inside a loop).

```
for (int k = 1; k <= 10; k++) {
    if (k % 2 == 0) continue;   // skip even numbers
    if (k > 7) break;           // stop completely after 7
    System.out.print(k + " ");
}
```

Output

```
1 3 5 7
```

Using `break` outside any loop or switch is a compile error: `error: break outside switch or loop`. Any code right after a `break` or `continue` in the same block can never run, so it is `error: unreachable statement`.

4.2 Labeled break and continue

With nested loops, a plain `break` or `continue` only affects the **innermost** loop. To control an outer loop, give it a **label** (a name followed by a colon) and use that name:

```
outer:
for (int r = 1; r <= 3; r++) {
    for (int c = 1; c <= 3; c++) {
        if (c == 2) continue outer;   // jump to the NEXT r
        if (r == 3) break outer;      // leave BOTH loops
        System.out.println("r=" + r + " c=" + c);
    }
}
```

Output

```
r=1 c=1
r=2 c=1
```

r	c	What happens
1	1	Neither condition is true: prints r=1 c=1.
1	2	`continue outer`: skips the rest of the inner loop, moves on to r = 2.
2	1	Prints r=2 c=1.
2	2	`continue outer`: moves on to r = 3.
3	1	`break outer`: both loops end.

> **Trainer's tip**
> A typical real use is searching a grid: once you find the item, `break outer` stops all the searching. Java has no `goto`; labels are the controlled replacement.

4.3 break vs return vs System.exit

Statement	What it stops
`break`	The current loop or switch only. The method continues after it.
`return`	The whole current method. In main, that ends the program normally.
`System.exit(0)`	The whole JVM, immediately. Avoid in real applications: it also kills everything else running.

5 Reading input with Scanner

5.1 Creating a Scanner

```
import java.util.Scanner;              // Scanner is in java.util, not java.lang

public class Hello {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);   // System.in = keyboard input
        System.out.print("Enter your name: ");
        String name = sc.nextLine();
        System.out.print("Enter your age: ");
        int age = sc.nextInt();
        System.out.println("Hi " + name + ", next year you will be " + (age + 1));
    }
}
```

The `import` line goes at the very top, before the class. Classes in `java.lang` (like String and System) are imported automatically; everything else must be imported. Imports and packages are covered properly in Part 8.

5.2 The main methods

Method	Reads	Example input → result
`nextInt()`	the next token as an int	42 → 42
`nextDouble()`	the next token as a double	3.5 → 3.5
`nextBoolean()`	the next token as a boolean	true → true
`next()`	the next **token** (one word, up to a space)	Mitansh Kanani → "Mitansh"
`nextLine()`	everything up to the end of the line	Mitansh Kanani → "Mitansh Kanani"
`hasNextInt()`	nothing! Only **checks** if the next token is a valid int	abc → false
`hasNext()`	nothing; checks if there is another token at all	—

A **token** is a piece of input separated by whitespace (spaces, tabs, Enter). `next()`, `nextInt()` and the other `nextX()` methods skip whitespace and read one token. `nextLine()` is different: it reads raw characters up to the Enter.

```
// input:  Mitansh Kanani here
//         3.5 true
String first = sc.next();      // "Mitansh"
String rest = sc.nextLine();   // " Kanani here"   (note the leading space!)
double d = sc.nextDouble();    // 3.5
boolean b = sc.nextBoolean();  // true
```

5.3 The nextInt() + nextLine() trap

```
System.out.print("Enter age: ");
int age = sc.nextInt();
System.out.print("Enter name: ");
String name = sc.nextLine();
System.out.println("Name = [" + name + "], age = " + age);
```

You type 25 and press Enter. The program never waits for the name!

```
Enter age: 25
Enter name: Name = [], age = 25
```

Input buffer after you type 25 [Enter] Mitansh [Enter]



After nextInt(): reads 25, STOPS before the Enter



Then nextLine(): reads up to that Enter → returns "" (empty)



When you press Enter, the keyboard sends `25\n`. `nextInt()` reads the digits and stops **before** the newline. `nextLine()` then sees that leftover newline straight away, thinks the line has ended, and returns an empty String.

Fix 1: throw away the rest of the line after reading a number:

```
int age = sc.nextInt();
sc.nextLine();                 // consume the leftover Enter
String name = sc.nextLine();   // now waits properly
```

Fix 2: read every input as a full line and convert it yourself. This avoids the problem completely:

```
int age = Integer.parseInt(sc.nextLine());
```

Your evaluator might ask

Q: Why does nextLine() return an empty string after nextInt()?

nextInt() reads only the number and leaves the newline character in the input buffer. The next nextLine() reads up to that newline and returns what is before it, which is nothing. Fix it by calling an extra nextLine() after nextInt(), or by reading whole lines and parsing them.

5.4 When the input is wrong

nextInt() but the user typed ten

```
Enter a number: ten
Exception in thread "main" java.util.InputMismatchException
    at java.base/java.util.Scanner.throwFor(Scanner.java:947)
    ...
    at S3.main(S3.java:7)
```

- `InputMismatchException`: the next token exists, but it is not the type you asked for.
- `NoSuchElementException`: there is no input left at all (for example, the input stream was closed or ended).

5.5 Validating input with hasNextInt()

```
int n;
while (true) {
    System.out.print("Enter a whole number: ");
    if (sc.hasNextInt()) {
        n = sc.nextInt();
        break;                                  // valid: leave the loop
    }
    System.out.println("'" + sc.next() + "' is not a whole number. Try again.");
}
System.out.println("You entered " + n);
```

Terminal

```
Enter a whole number: abc
'abc' is not a whole number. Try again.
Enter a whole number: 12.5
'12.5' is not a whole number. Try again.
Enter a whole number: 42
You entered 42
```

Common mistake

`hasNextInt()` only **looks** at the next token; it doesn't remove it. If you forget `sc.next()` in the invalid branch, the bad token stays there forever, hasNextInt() keeps returning false, and you get an **infinite loop**.

You can also use `hasNextInt()` to read numbers until the input runs out:

```
int sum = 0, count = 0;
while (sc.hasNextInt()) {     // input "4 5 6" then "7 x": stops at x
    sum += sc.nextInt();
    count++;
}
System.out.println(count + " numbers, sum " + sum);   // 4 numbers, sum 22
```

5.6 Closing the Scanner

You may see `sc.close()` in tutorials. Be careful: closing a Scanner also closes `System.in`, and System.in **cannot be reopened**. Any new Scanner on System.in then fails with `NoSuchElementException`.

> **Trainer's tip**
> Create **one** Scanner for System.in at the start of the program and use it everywhere. Don't close it, or close it only at the very end. (Scanners on files are different, and must be closed. You will see that in Part 18.)

5.7 The menu pattern: do-while + switch + Scanner

```
Scanner sc = new Scanner(System.in);
int choice;
do {
    System.out.println("1. Say hi");
    System.out.println("2. Exit");
    System.out.print("Choice: ");
    choice = sc.nextInt();
    switch (choice) {
        case 1 -> System.out.println("Hi!");
        case 2 -> System.out.println("Bye!");
        default -> System.out.println("Invalid choice");
    }
} while (choice != 2);
```

Notice that `choice` is declared **before** the do-while. The condition `choice != 2` is outside the braces, so a variable declared inside the body would be out of scope there. This pattern is the skeleton of every console application, including a banking system.

6 Classic loop programs

These patterns come up again and again in assignments, interviews and coding tests. Learn the **technique**, not the code.

6.1 Taking a number apart: % 10 and / 10

```
int n = 1234, sum = 0, rev = 0, t = n;
while (t > 0) {
    int digit = t % 10;        // last digit
    sum += digit;
    rev = rev * 10 + digit;    // append digit to the reverse
    t /= 10;                   // remove last digit
}
// sum = 10, rev = 4321
```

Round	t at start	digit = t % 10	sum	rev = rev*10 + digit	t / 10
1	1234	4	4	4	123
2	123	3	7	43	12

Round	t at start	digit = t % 10	sum	rev = rev*10 + digit	t / 10
3	12	2	9	432	1
4	1	1	10	4321	0 → loop ends

Trainer's tip

Work on a copy (`t`), not on `n` itself. The loop destroys the number, and you usually still need the original, for example to check if it is a palindrome (`rev == n`).

6.2 Prime check

```
boolean prime = n > 1;               // 0 and 1 are not prime
for (int k = 2; k * k <= n; k++) {
    if (n % k == 0) {
        prime = false;
        break;                          // one divisor is enough
    }
}
```

Why only up to √n (`k * k <= n`)? If n = a × b, one of a or b must be at most √n. So if there is no divisor up to √n, there is none at all. For n = 1,000,003 that is about 1,000 checks instead of a million.

6.3 Factorial, Fibonacci and GCD

Factorial

```
long f = 1;
for (int k = 2; k <= 20; k++) f *= k;       // 20! = 2432902008176640000

int fact = 1;
for (int k = 2; k <= 13; k++) fact *= k;    // 13! as int = 1932053504   WRONG (overflow)
```

13! is 6,227,020,800, more than an int can hold. As you know from Part 2, overflow gives no error. Use long: it holds up to 20!. Beyond that, even long overflows.

Fibonacci: each number is the sum of the previous two

```
int a = 0, b = 1;
for (int k = 0; k < 10; k++) {
    System.out.print(a + " ");     // 0 1 1 2 3 5 8 13 21 34
    int next = a + b;
    a = b;
    b = next;
}
```

GCD with Euclid's algorithm: gcd(48, 18) = gcd(18, 12) = gcd(12, 6) = gcd(6, 0) = 6

```
int x = 48, y = 18;
while (y != 0) {
    int tmp = y;
    y = x % y;
    x = tmp;
}
// x = 6
```

6.4 Patterns: thinking in rows and columns

For any star pattern, the outer loop is the **row**, and the inner loop(s) print what is in that row. First write down, for row r, how many spaces and how many stars it has. The code follows from that table.

```
int n = 4;
for (int r = 1; r <= n; r++) {
    for (int s = 1; s <= n - r; s++) System.out.print(" ");       // n - r spaces
    for (int s = 1; s <= 2 * r - 1; s++) System.out.print("*");   // 2r - 1 stars
    System.out.println();                                         // end of row
}
```

Row r	Spaces (n - r)	Stars (2r - 1)	Printed
1	3	1	*
2	2	3	***
3	1	5	*****
4	0	7	*******

7 Common errors in this part (real messages)

You wrote	You see	Fix
`if (x = 5)`	`incompatible types: int cannot be converted to boolean`	Use == to compare.
`if (done = true)`	No error. Always true!	Write if (done).
`switch on a long`	`selector type long is not allowed`	Use int, or if-else.
`case y: (y is a normal variable)`	`constant expression required`	Use a literal or a final constant.
`Same case twice`	`duplicate case label`	Remove the duplicate.
`Mixing case X: and case X ->`	`different case kinds used in the switch`	Use one style.
`Switch expression with no default`	`the switch expression does not cover all possible input values`	Add default.
`Block in a switch expression without yield`	`switch rule completes without providing a value`	Add yield value;
`Code after while(true) {}`	`unreachable statement`	Add a break, or move the code.
`Code after break in the same block`	`unreachable statement`	Remove or move it.
`break in plain code`	`break outside switch or loop`	Only inside a loop or switch.

You wrote	You see	Fix
`Using the for variable after the loop`	`cannot find symbol symbol: variable i`	Declare it before the loop.
`} while (i > 0) without ;`	`';' expected`	do-while ends with a semicolon.
`switch on a null String`	`NullPointerException` (runtime)	Check for null first.
`nextInt() but the user typed text`	`InputMismatchException` (runtime)	Validate with hasNextInt().
`Reading after the input ended`	`NoSuchElementException` (runtime)	Don’t close System.in; check hasNext().

Common mistake

As in Part 2, the worst bugs here give no error: missing braces, the semicolon after if or for, missing break (fall-through), boolean assignment in if, the dangling else, off-by-one loops and the nextInt/nextLine trap. Trace your code; don’t trust indentation.

8 Self-check questions

Answer without looking, then check. If you miss one, re-read the section in brackets.

#	Question	Answer
1	Why does if (x = 5) fail to compile when x is an int, but if (done = true) compiles?	x = 5 produces an int, and conditions must be boolean. done = true produces a boolean, so it compiles and is always true. (1.1)
2	In an else-if ladder, how many branches can run?	At most one: the first one whose condition is true. (1.2)
3	What does an else attach to when there are no braces?	The nearest unmatched if (the dangling else). (1.5)
4	What does if (x > 5); { ... } do?	The ; is an empty statement controlled by the if. The block always runs. (1.4)
5	What is fall-through?	Without break, execution continues into the following cases until a break or the end of the switch. (2.2)
6	Which types can a switch use? Name two that can’t.	byte, short, char, int (and wrappers), String, enum. Not long, float, double, boolean. (2.3)
7	Why must case labels be constants?	The switch is compiled into a jump table (tableswitch/lookupswitch) built at compile time. (2.3)
8	What happens when you switch on a null String?	NullPointerException: the switch calls hashCode() on it. (2.3)
9	Two differences between arrow cases and colon cases?	No fall-through, and several labels separated by commas. (2.4)
10	What is yield for?	To give the value of a block branch in a switch expression. (2.5)

#	Question	Answer
11	Why must a switch expression on an int have a default?	A switch expression must cover every possible input value. (2.5)
12	Difference between while and do-while?	while checks first (0 or more runs); do-while checks after (at least 1 run). (3.1, 3.2)
13	In what order do the parts of a for loop run?	init once, then condition → body → update, repeated. (3.3)
14	Why is while(false) an error but if(false) is not?	The JLS makes an exception for if, so constants can switch code on and off. Loops have no exception. (3.4)
15	How many times does for (double x = 0; x < 1.0; x += 0.1) run?	11. After 10 additions, x is 0.9999999999999999. (3.6)
16	Why is for (byte b = 0; b < 200; b++) infinite?	A byte can never reach 200; after 127 it wraps to -128. (3.6)
17	Difference between break and continue?	break exits the loop; continue skips to the next round. (4.1)
18	How do you exit two nested loops at once?	Label the outer loop and use break label;. (4.2)
19	Difference between next() and nextLine()?	next() reads one token (up to whitespace); nextLine() reads the rest of the line. (5.2)
20	Why does nextLine() return "" after nextInt()?	nextInt() leaves the newline in the buffer; nextLine() reads up to it. (5.3)
21	What happens if you don’t call sc.next() after hasNextInt() returns false in a loop?	The bad token is never removed, so it’s an infinite loop. (5.5)
22	Why shouldn’t you close a Scanner on System.in in the middle of a program?	It closes System.in, which cannot be reopened. (5.6)
23	Why does the prime check only go up to k * k <= n?	If n has a divisor, one of the pair is at most √n. (6.2)
24	Why is 13! wrong in an int?	It is 6,227,020,800, which overflows int silently. Use long. (6.3)

9 Cheat sheet

Part 3 at a glance

```
IF          if (cond) {...} else if (cond) {...} else {...}    always use braces
            condition must be boolean     if (done) not if (done == true)
            first true branch wins         else -> nearest unmatched if

SWITCH      types: byte short char int (+wrappers), String, enum
            NOT allowed: long, float, double, boolean
            case labels: compile-time constants, no duplicates
            case X:   falls through until break
            case X -> no fall-through;  case 6, 7 -> several labels
EXPRESSION String s = switch (d) { case 1 -> "Mon"; default -> "?"; };
            must cover all values (default);  block -> { ...; yield value; }

LOOPS       while (cond) {...}           0 or more rounds
            do {...} while (cond);       1 or more rounds   (semicolon!)
            for (init; cond; update)     init once, then cond -> body -> update
            for (;;) / while (true)      infinite: needs break or return
TRAPS       double counter (0.1 loop runs 11x) | byte counter wraps | off-by-one (< vs <=)
            ; after if/for/while header | code after while(true) or break = unreachable

JUMPS       break: exit loop/switch    continue: next round    label: outer: ... break outer;
            return: exit method        System.exit(0): kill JVM (avoid)

SCANNER     import java.util.Scanner;   Scanner sc = new Scanner(System.in);  (one)
            nextInt nextDouble nextBoolean next = one token     nextLine = rest of line
            after nextInt(): call sc.nextLine() before reading a line
            validate: if (sc.hasNextInt()) n = sc.nextInt(); else sc.next();
                      (sc.next() discards the bad token, or you loop forever)
            errors: InputMismatchException (wrong type), NoSuchElementException (no input)
            don't close System.in in the middle of the program

DIGITS      d = n % 10 (last digit)    n /= 10 (drop it)    rev = rev * 10 + d
PRIME       for (k = 2; k * k <= n; k++)    FACTORIAL: long (int overflows at 13!)
```

10 Assignments

Same rules: no AI, no copy-paste, compile and run from the terminal, everything in `part-03`, one-line comment at the top of each file. For prediction tasks, predict **first**, then run, then explain every wrong one.

Before you start

A3 and A8 use the input-validation loop from Section 5.5. You will need it more than once in A8; copy it for now. In Part 5 you will learn to put it in a method and reuse it.

A1. GradeCalculator.java (Easy)

Read marks (a whole number) with Scanner. If it is outside 0–100, print `Invalid marks`. Otherwise print the grade using an else-if ladder: A for 90+, B for 75+, C for 60+, D for 40+, F below 40. Then print Pass (40+) or Fail using the **ternary operator**.

```
Terminal

> java GradeCalculator
Enter marks: 76
Grade: B
Result: Pass
```

Test the edges and list your results in a comment: 100, 90, 89, 40, 39, 0, -5 and 101. Boundary values are where ladder bugs hide.

A2. DayFinder.java (Easy)

Read a day number 1–7 (1 = Monday). Print the day's name **three** ways:

1. A classic switch with `case 1:` and `break`.
2. An arrow switch statement (`case 1 ->`).
3. A switch **expression** whose result is stored in a String variable, then printed.

Then use another switch expression with comma-separated labels to print `Weekday` or `Weekend`. Any other number prints `Invalid day`.

Then break it on purpose: remove one `break` from version 1, run it, and write in a comment exactly what printed and why. Put the break back afterwards.

A3. NumberAnalyzer.java (Medium)

Read a positive whole number. Keep asking until the input is a valid int **and** greater than 0. Then print its analysis. Use while loops with `% 10` and `/ 10`. **Don't use Math.pow or Strings**: compute powers with a loop.

```
Terminal

> java NumberAnalyzer
Enter a positive number: abc
'abc' is not a whole number.
Enter a positive number: -5
Must be greater than 0.
Enter a positive number: 153
Digits     : 3
Digit sum  : 9
Reverse    : 351
Palindrome : false
Prime      : false
Armstrong  : true
```

An **Armstrong** number equals the sum of its digits, each raised to the power of the number of digits: 153 = $1^3 + 5^3 + 3^3$. Also test with 12321 (a palindrome) and 7.

A4. PatternPrinter.java (Medium)

Read n and print these five patterns, with a blank line between them. Before coding each one, write the "row r → spaces, stars" table from Section 6.4 as a comment. Output for n = 4:

Shown side by side here; print them one below the other

```
*                * * * *             *             1             1
* *              * * *              ***            1 2           2  3
* * *            * *              *****            1 2 3         4  5  6
* * * *          *              *******            1 2 3 4       7  8  9  10
```

Each star is followed by a space in patterns 1 and 2; the pyramid has no spaces between stars. Pattern 5 (Floyd’s triangle) keeps counting across rows. Use `%-3d` to keep its columns straight.

A5. LoopTrace.java (Medium)

Put each snippet in main. For each one, write your **predicted** output as a comment first, then run. For every wrong prediction, explain why in a comment.

Snippets 1 to 5

```
// 1
int i = 5;
do { System.out.print(i + " "); i++; } while (i < 5);

// 2
int count = 0;
for (double x = 0.0; x < 1.0; x += 0.1) count++;
System.out.println(count);

// 3
for (int k = 0; k < 3; k++); { System.out.println("Hi"); }

// 4
int x = 2;
switch (x) {
    case 1: System.out.print("one ");
    case 2: System.out.print("two ");
    case 3: System.out.print("three "); break;
    case 4: System.out.print("four ");
}

// 5
int a = 5, b = 15;
if (a > 10)
    if (b > 10)
        System.out.println("both big");
else
    System.out.println("a is small");
System.out.println("end");
```

Snippets 6 to 10

```
// 6
outer:
for (int r = 1; r <= 3; r++) {
    for (int c = 1; c <= 3; c++) {
        if (c > r) continue outer;
        if (r + c == 5) break outer;
        System.out.print(r + "" + c + " ");
    }
}

// 7
int n = 0;
while (n++ < 3) System.out.print(n + " ");
System.out.println(n);

// 8
int total = 0;
for (int j = 1; j <= 10; j++) { if (j % 3 == 0) continue; total += j; }
System.out.println(total);

// 9
int m = 3;
String s = switch (m) {
    case 1, 2 -> "low";
    case 3 -> { String t = "mid"; yield t + "!"; }
    default -> "high";
};
System.out.println(s);

// 10
int y = 10;
if (y > 5) System.out.print("A");
else if (y > 8) System.out.print("B");
if (y > 8) System.out.print("C");
```

A6. GuessingGame.java (Medium)

The computer picks a secret number from 1 to 100: `int secret = (int) (Math.random() * 100) + 1;` (`Math.random()` returns a double from 0.0 up to, but not including, 1.0).

- The player has **7 attempts**. After each guess print `Too high` or `Too low`.
- Invalid input (not a number, or outside 1–100) prints a message and does **not** use up an attempt.
- On a correct guess, print `Got it in N attempts!` and stop asking (use `break`).
- If all 7 attempts are used, print `Out of attempts. The number was X.`.
- Afterwards, ask `Play again? (y/n)` and repeat the whole game with a do-while.

In a comment, explain why 7 attempts are always enough with the right strategy. (Hint: each guess can halve the range; $2^7 = 128$.)

A7. PrimeRange.java (Medium)

Read two numbers a and b. Print every prime between them (inclusive), then the count. **Rule:** no boolean flag variable is allowed. Use a **labeled continue** to skip a number as soon as you find a divisor.

Terminal

```
> java PrimeRange
Enter start: 10
Enter end: 50
11 13 17 19 23 29 31 37 41 43 47
Count: 11
```

Make sure 0 and 1 are not printed as primes (try a = 0), and handle a > b by swapping them.

A8. MiniATM.java (Medium–Hard)

A menu-driven ATM, the first piece of a banking system. Balance starts at Rs 10000 (use `int`; remember: never double for money). Use do-while + an arrow switch.

- **1. Check balance**, **2. Deposit**, **3. Withdraw**, **4. Exit**.
- Typing text instead of a number must **never crash** the program, for the menu choice or for amounts.
- Deposit: amount must be positive.
- Withdraw: amount must be positive, not more than the balance, and a multiple of 100.
- Count successful transactions; print the count on exit. Any other choice prints `Invalid option. Choose 1 to 4.`.

Part of a sample run (after a deposit of 500)

```
===== MINI ATM =====
1. Check balance
2. Deposit
3. Withdraw
4. Exit
Choose: 3
Amount to withdraw: 20000
Insufficient balance. Available: Rs 10500
...
Choose: 3
Amount to withdraw: abc
Please enter a whole number.
Amount to withdraw: 1000
Please collect your cash. New balance: Rs 9500
...
Choose: 4
Thank you! Transactions done: 2
```

Carried over from Part 2

Full details are in the Part 2 PDF, Section 12. Short versions:

C1. IncrementPuzzle.java (Pending, Part 2 A5)

Predict, run, then trace each value step by step in comments:

```
int a = 5;
int b = a++ + a++;
int c = ++a + ++a;
int d = a-- - --a;
int e = 10;  e = e++ + ++e;
int g = 3;   g += g++ * 2;
int h = 2;   h = h++ * ++h;
```

C2. BitwiseToolkit.java **(Pending, Part 2 A6)**

Analyse n with bitwise operators only (even check, binary, ×8, ÷4, power of two, ~n). Then the READ/WRITE/PRINT/SHARE permission flags: start with READ and SHARE, grant PRINT, revoke READ, print 12 and 1100.

C3. SwapThreeWays.java **(Pending, Part 2 A7)**

Swap two ints with a temp variable, with + and -, and with ^. Run it with 2147483647 1 and explain why the overflowing version still works.

C4. ShortCircuitProof.java **(Pending, Part 2 A8)**

Use a counter to prove && and || skip the right side, while & and | don’t. Then print the largest of three numbers with a nested ternary, and whether it is even or odd.

C5. NumberTraps.java **(Optional, Part 2 A3)**

Overflow (30 days in ms, Math.abs(Integer.MIN_VALUE)), money with long paise (Rs 3.30), and ending with Math.multiplyExact throwing ArithmeticException.

What happens next

Push Part 3 when you are done and send the repo link. I’ll run everything, including the Scanner programs with tricky input, and list only the mistakes worth fixing.